

Repair Shop Management App

Tech Stack · Architecture · Build Phases · Workflow · Tools

Nilakshith Enterprise · Barak Valley, Assam · June 2025

1. Tech Stack

Frontend — Mobile Apps (Receptionist & Technician)

React Native with Expo is the ideal choice. One codebase runs on both Android and iOS, which matters since your staff will have different phones. Expo provides camera access (for attendance selfies and onsite photos), push notifications, GPS coordinates, and Bluetooth printing — all required by the spec — without needing separate native iOS/Android developers.

Frontend — Admin Panel

React.js (web) for the full admin dashboard. Admins work on desktops and need rich data tables, charts, filtering, and reports. Deploy to Vercel for free. A React Native wrapper can optionally be added later for mobile admin access.

Backend

Supabase is the perfect fit for this project. It gives you: PostgreSQL database, authentication with role-based access control, file storage (for selfies and invoices), real-time subscriptions (all dashboards update live when a job status changes), and auto-generated REST APIs — all on a generous free tier. This eliminates most custom backend work. For complex business logic (salary calculation, advanced reports), add Supabase Edge Functions (serverless, free tier) or a small Node.js + Express server on Railway (free tier).

Core Database Tables (PostgreSQL via Supabase)

Table	Key Columns
users	id, name, role (admin/receptionist/technician), email, phone, is_active, created_at
jobs	job_id, customer_name, contact, device_type, issue, remarks, technician_id, status, priority, job_type, created_at
job_materials	id, job_id, material_name, quantity, unit_cost, total_cost
attendance	id, user_id, date, check_in_time, check_out_time, photo_url, gps_lat, gps_lng, status (present/halfday/leave/absent)
inventory	id, item_name, quantity, unit, low_stock_threshold, last_updated
billing	id, job_id, parts_total, labour_charge, tax_percent, discount, grand_total, paid, invoice_url
payments	id, type (advance/expenditure/materials/office), amount, date, description, user_id
salary	id, user_id, month, base_days, present_days, halfday_count, leave_count, ot_hours, early_hours, advance_deducted, net_salary

notifications	id, job_id, recipient_type, channel (push/whatsapp/email), message, sent_at, status
---------------	---

External Integrations

Need	Tool / Service	Cost
WhatsApp messages to customers	Twilio WhatsApp API or WATI (India-friendly)	Free trial, then paid per message
Push notifications (device alerts)	Expo Push Notifications via FCM (Android) / APNs (iOS)	Free
Email — invoices and account activation	Resend or SendGrid	Free tier (100/day)
PDF invoice generation	jsPDF (web admin) / react-native-html-to-pdf (mobile)	Free open-source
GPS technician tracking	Google Maps API + Expo Location	Free quota, then paid
Bluetooth receipt printing	react-native-bluetooth-escpos-printer library	Free open-source

2. System Architecture

Layer	What It Does
Frontend (Mobile)	React Native (Expo) for Receptionist and Technician apps. Expo Go for development, EAS Build for production distribution.
Frontend (Web Admin)	React.js for the admin panel. Hosted on Vercel (free). Full data tables, charts, reports, and salary management.
API Layer	Supabase auto-generated REST API handles 80% of operations. Supabase Edge Functions handle custom logic. Optionally Node.js/Express on Railway for heavy processing.
Auth Layer	Supabase Auth with JWT tokens. Row Level Security (RLS) policies on every table enforce that each role only sees what they are permitted to see.
Database	PostgreSQL via Supabase — single source of truth for all jobs, users, inventory, billing, salary, and attendance.
Realtime	Supabase Realtime (WebSockets) — job status changes push instantly to all connected dashboards with zero polling.
Storage	Supabase Storage — attendance selfies, onsite photos, and generated PDF invoices stored as blobs with signed access URLs.
Notifications	Expo Push for in-device alerts. Twilio/WATI for WhatsApp. Resend for email. All triggered via Supabase database webhooks or Edge Functions.

Hosting	Admin web: Vercel (free). Custom API (if needed): Railway or Render (free tier). Mobile: Expo EAS Build → Google Play Store + Apple App Store.
---------	---

3. Core Operational Workflow

The system follows four actors: Receptionist, Technician, Admin, and Customer. Here is the complete step-by-step operational flow:

Step	Actor	Action
1	Receptionist	Customer arrives. Receptionist opens the Customer Intake Form and fills in: customer name, contact number, device type, reported issue, remarks, job type (Inhouse/Onsite), and priority (Normal/High/Urgent).
2	Receptionist	Selects an available technician from the dropdown and submits the form. The system auto-generates a unique Job ID with a timestamp.
3	Receptionist	A receipt is printed via Bluetooth thermal printer for the customer, showing Job ID, device details, and contact info.
4	System	Automatic notifications fire: push notification to the assigned technician, WhatsApp message to the customer confirming job receipt, and an in-app alert to the admin.
5	Technician	Sees the new job in the Assigned Jobs List. High/Urgent jobs are prominently flagged. Reviews all job details.
6	Technician	For Onsite jobs: travels to the customer location and takes a selfie on arrival. Expo Camera captures GPS coordinates automatically — this creates the 'site visit start' record.
7	Technician	Works on the device. Records all materials/parts used (name, quantity, cost), writes work notes (diagnosis and steps taken), and updates job status to In Progress.
8	System	Notification fires to receptionist and admin when the technician updates status.
9	Technician	For Onsite jobs: takes a second selfie before leaving the location. This closes the site visit record with GPS and timestamp (used for TA calculation).
10	Technician	Marks the job as Completed. System notifies the receptionist automatically.
11	Receptionist	Sends a WhatsApp 'Ready for Pickup' message to the customer directly from the app.
12	Receptionist	Generates the final bill. System auto-pulls: customer details, technician info, all materials used (from job_materials table), applies labour charges, tax, and any discounts.
13	Receptionist	Invoice is printed and/or shared via WhatsApp/email with the customer. Payment is recorded in the payments table.
14	Admin	Monitors the full job board in real time, tracks technician GPS locations, reviews inventory levels, and checks daily financial summaries.

4. Development Phases & Timeline

Phase 1	Project Setup & Auth	4–6 days
Set up Supabase project, design the complete PostgreSQL schema, configure authentication with three roles (admin, receptionist, technician), and scaffold the Expo mobile project with navigation structure (drawer or tab nav for each role). • Tools: Supabase dashboard (supabase.com), VS Code, Expo CLI, GitHub for version control • Vibe coding: Bolt.new — prompt it to generate schema SQL and Supabase config; then Cursor for integration • Key output: All database tables created, auth working, role-based screen routing in the app <i>Start here: Create the schema in dbdiagram.io first, review all relationships, then export SQL to Supabase.</i>		
Phase 2	Attendance Module	3–4 days
Selfie-based attendance for both receptionist and technician. The device camera opens, the selfie is taken, GPS coordinates are captured, the image uploads to Supabase Storage, and a record is written with user_id, timestamp, photo_url, gps_lat, and gps_lng. • Tools: Expo Camera, Expo Location, Supabase Storage, Supabase JS client • Vibe coding: Cursor — prompt with the full attendance schema and camera/GPS requirements • Features: Check-in/check-out, leave marking, half-day marking, view previous attendance with photos • Key output: Punch-in/out working with photo and GPS capture on both roles; admin can view all attendance		
Phase 3	Receptionist Dashboard	8–10 days
The most complex module. Build in this order to avoid rework: Customer Intake Form first, then Job ID generation, then technician assignment dropdown, then the job tracking list view, then priority highlighting, and finally receipt printing. • Tools: React Native, Supabase JS client, Expo Print (or react-native-bluetooth-escpos-printer) • Vibe coding: Lovable.dev or Bolt.new for the form UI and job list cards; then Cursor for Supabase integration • Features: Intake form, auto Job ID, technician dropdown, job list with status colours, priority badges, reassign technician, print receipt • Key output: Full job creation, assignment, and tracking flow working end-to-end		
Phase 4	Technician Dashboard	6–7 days
Assigned jobs list with all job details, job detail view, materials/parts entry form (name, qty, cost), work notes field, status update buttons (In Progress, Waiting for Materials, Completed), and onsite selfie capture before and after visiting a location. • Tools: React Native, Supabase Realtime, Expo Camera, Expo Location • Vibe coding: Cursor — status updates are CRUD operations; Cursor handles these well with clear schema context • Features: Filtered job list, material logging, work notes, status updates, onsite selfie with GPS, TA tracking • Key output: Technician can fully manage jobs, update status, and log all materials from their phone		

Phase 5 Notifications System

5–6 days

Three notification channels working together: in-app via Supabase Realtime (instant, no delay), push notifications via Expo Push / FCM / APNs (for background alerts), and WhatsApp messages via Twilio or WATI for customer-facing updates.

- Tools: Expo Notifications, Supabase Edge Functions, Supabase database webhooks, Twilio / WATI API
- Vibe coding: Cursor — write Edge Functions triggered by database INSERT/UPDATE events on the jobs table
- Triggers: Job created, status changed to In Progress, status changed to Completed, Ready for Pickup sent
- Key output: All three parties (customer, technician, admin) automatically notified at every key stage

Important: Start Twilio WhatsApp sandbox registration and APNs certificate setup early — these can take several days to approve.

Phase 6 Admin Panel (Web)

10–12 days

React web app. Build in sub-sections: user management first (approve/block accounts via email activation), then job oversight board, then inventory management with low-stock alerts, then technician performance reports, then salary management, then advance salary, then daily expenditure logging.

- Tools: React.js, Recharts (for bar/line charts), Supabase JS client, Vercel for deployment
- Vibe coding: v0.dev by Vercel is the best free tool for React admin dashboards — generates tables, charts, sidebars
- Features: User CRUD, job board with filters, inventory tracker, performance charts, salary auto-calc, expenditure log, billing config
- Key output: Admin has full visibility over all operations, staff, inventory, and finances from a web browser

Phase 7 Billing & Invoice System

5–6 days

Auto-pull customer name, job ID, technician name, all materials used with quantities and costs (from job_materials), apply labour charges, tax percentage, and any discounts. Generate a formatted PDF invoice. Offer Print, Email, and WhatsApp share options.

- Tools: jsPDF (web admin), react-native-html-to-pdf (mobile), Resend for email delivery
- Vibe coding: Cursor — describe the invoice layout, data sources, and calculation formula precisely in the prompt
- Features: Itemised materials list, labour charge, tax, discount, grand total, print/email/WhatsApp delivery
- Key output: One-click professional invoice generation and multi-channel delivery to the customer

Phase 8	Salary & Money Management	6–7 days
Auto-calculate monthly salary from the attendance table. Account for: present days (full pay), half days (half pay), leave days (no pay or leave encashment), OT hours (OT rate x hours), early departure hours (deduction), and advance salary installment deductions. - Tools: Supabase Edge Functions for calculation logic, React Native for salary view, Expo Print for salary slip - Vibe coding: Cursor — describe the exact salary formula with all edge cases before prompting; be very precise - Features: Monthly salary auto-calc, advance salary (with notification + print receipt), daily expenditure log, materials purchase log, office development expense log - Key output: Admin can process payroll, issue advance salary with receipts, and track all outgoing money <i>Critical: Map the entire salary formula on paper first — base rate, OT rate, halfday deduction percentage, advance installment logic — before writing any code. Wrong formula = wrong payroll.</i>		

Phase 9	Testing, Polish & Deployment	8–10 days
End-to-end testing of all user flows across all three roles. Test all notification triggers, salary calculation edge cases (e.g. month with a public holiday, advance deducted across multiple months), and billing scenarios. Fix edge cases. Then deploy. - Mobile: EAS Build (Expo) to create production APK/IPA; submit to Google Play Store and Apple App Store - Web admin: Deploy to Vercel with one command (vercel --prod); set Supabase environment variables in Vercel dashboard - Test on real devices, not just simulators, especially for camera, GPS, Bluetooth printing, and push notifications - Key output: All three apps live in production and accessible to real users		

5. Timeline Summary		
Phase	Feature	Estimate
1	Project Setup & Auth	4–6 days
2	Attendance Module	3–4 days
3	Receptionist Dashboard	8–10 days
4	Technician Dashboard	6–7 days
5	Notifications System	5–6 days
6	Admin Panel (Web)	10–12 days
7	Billing & Invoice System	5–6 days
8	Salary & Money Management	6–7 days
9	Testing, Polish & Deployment	8–10 days

	TOTAL	~55–68 days
--	-------	-------------

6. Free Vibe Coding Platforms

Platform	Best For	Free?
Bolt.new	Full-stack scaffolding, initial project setup, multi-field forms, CRUD screens. Good starting point for any phase.	Free tier (limited tokens/day)
Lovable.dev	Mobile-style UI, card layouts, and form screens with good visual quality. Ideal for the Receptionist and Technician dashboards.	Free tier
v0.dev (Vercel)	React admin dashboards, data tables, charts, sidebars. Best tool for generating the Admin Panel web UI.	Free
Cursor	Deep code editing, Supabase integration, business logic, Edge Functions. The main workhorse for integration work.	Free tier (limited)
GitHub Copilot	Line-by-line autocomplete while writing code in VS Code. Works well alongside Cursor.	Free for students
Claude.ai	Architecture decisions, master prompts for each phase, complex logic design, salary formula design, schema review.	Free tier

Recommended Workflow

Use Bolt.new or Lovable to generate the initial UI structure and screen layouts for each module. Bring that generated code into Cursor for Supabase integration and business logic. Use Claude to write detailed master prompts for each phase — the more precise the prompt, the better the output. Never prompt vague instructions like 'build an attendance system'; instead write the full schema, the required fields, the Supabase tables involved, and the exact user flow before hitting generate.

7. Critical Advice Before You Start

Design the database schema first

Spend 2–3 days designing your Supabase schema on paper or in dbdiagram.io (free) before writing any code. Every feature depends on the data model being correct from day one. A wrong schema means rebuilding entire modules later. Prioritise: jobs, attendance, job_materials, users, and salary — get the relationships and foreign keys between these right before anything else.

Set up Row Level Security (RLS) in Phase 1

Configure Supabase RLS policies early. This ensures: a technician cannot see another technician's jobs or salary, a receptionist cannot access salary records, and only admin can approve new user registrations. Adding RLS retroactively after data is in the system is painful and error-prone.

Use Supabase Realtime from day one

Subscribe to the jobs table in all three dashboards from the beginning. This gives you live job status updates without any polling. It is what makes the app feel professional and connected. Supabase Realtime is free, and the setup is just a few lines of code.

Prototype notifications before going deep

Test one WhatsApp message and one push notification in Phase 5 before building the full notification logic. WhatsApp sandbox registration (Twilio) and APNs certificates (iOS) can take several days to get approved — start the approval process early, ideally in Phase 1.

Auto-generate Job IDs server-side

Never generate Job IDs in the mobile app. Use a Supabase PostgreSQL sequence or a database function with a custom prefix (e.g. NE-2025-0001) to generate them server-side. This guarantees uniqueness even when two receptionists create jobs at the same time.

Design the salary formula on paper first

The salary calculation module (Phase 8) is the most logic-heavy part of the system. Before writing any code, write out the complete formula: base daily rate, OT rate per hour, halfday percentage, leave deduction rules, advance installment calculation. Have the business owner review and sign off on the formula. Bugs in payroll are the most damaging bugs in any staff management system.

Test on real devices early

The camera, GPS, Bluetooth printing, and push notifications all behave differently on real Android and iOS devices than they do in the Expo Go simulator. Start testing on a real phone in Phase 2 (Attendance) and keep doing so through every phase. Never leave real-device testing until the end.